

DBMS LAB ASSIGNMENT

Labs 4 to 6

Functions | Operators | Special and Set Operations

Name: Sparsh
Roll No / Reg No: _____
Course: BCA, R.V. University
Database used: MySQL / MariaDB 10.11

Lab 4	Implement SQL Functions (Conversion and Date Functions)
Lab 5	Implement SQL Operators (Arithmetic, Comparison and Logical)
Lab 6	Implement SQL Operators (Special Operators and Set Operations)

LAB 4: Implement SQL Functions (Conversion and Date Functions)

Aim: To implement SQL functions: conversion functions and date functions.

1. Create database, table and insert data

INPUT (SQL)

```
DROP DATABASE IF EXISTS lab4;
CREATE DATABASE lab4;
USE lab4;
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer VARCHAR(20),
  amount VARCHAR(10),
  order_date DATE
);
INSERT INTO orders VALUES
(1, 'Asha', '1250.75', '2026-01-15'),
(2, 'Ravi', '890.50', '2026-03-02'),
(3, 'Meera', '2300.00', '2026-07-28'),
(4, 'Karan', '450.25', '2026-10-05');
SELECT * FROM orders;
```

OUTPUT

order_id	customer	amount	order_date
1	Asha	1250.75	2026-01-15
2	Ravi	890.50	2026-03-02
3	Meera	2300.00	2026-07-28
4	Karan	450.25	2026-10-05

2. CONVERSION: CAST - string to number and number to string

INPUT (SQL)

```
SELECT order_id,
       amount,
       CAST(amount AS DECIMAL(10,2)) AS amount_decimal,
       CAST(amount AS SIGNED) AS amount_int,
       CAST(order_id AS CHAR) AS id_as_text
FROM orders;
```

OUTPUT

order_id	amount	amount_decimal	amount_int	id_as_text
1	1250.75	1250.75	1250	1
2	890.50	890.50	890	2
3	2300.00	2300.00	2300	3
4	450.25	450.25	450	4

3. CONVERSION: CONVERT - number and date conversion

INPUT (SQL)

```
SELECT CONVERT('123', UNSIGNED) AS text_to_number,
       CONVERT(98.76, CHAR) AS number_to_text,
       CONVERT('2026-12-25', DATE) AS text_to_date;
```

OUTPUT

text_to_number	number_to_text	text_to_date
123	98.76	2026-12-25

4. CONVERSION: STR_TO_DATE - text in any format to a date

INPUT (SQL)

```
SELECT STR_TO_DATE('25-12-2026', '%d-%m-%Y') AS d1,  
       STR_TO_DATE('March 5, 2026', '%M %d, %Y') AS d2;
```

OUTPUT

d1	d2
2026-12-25	2026-03-05

5. CONVERSION: DATE_FORMAT and FORMAT - date/number to formatted text

INPUT (SQL)

```
SELECT order_id,  
       DATE_FORMAT(order_date, '%d-%b-%Y') AS short_date,  
       DATE_FORMAT(order_date, '%W, %M %e, %Y') AS long_date,  
       FORMAT(CAST(amount AS DECIMAL(10,2)) * 1000, 2) AS amount_formatted  
FROM orders;
```

OUTPUT

order_id	short_date	long_date	amount_formatted
1	15-Jan-2026	Thursday, January 15, 2026	1,250,750.00
2	02-Mar-2026	Monday, March 2, 2026	890,500.00
3	28-Jul-2026	Tuesday, July 28, 2026	2,300,000.00
4	05-Oct-2026	Monday, October 5, 2026	450,250.00

6. CONVERSION: implicit conversion (string used in arithmetic)

INPUT (SQL)

```
SELECT '10' + 5 AS implicit_add, '3.5' * 2 AS implicit_mul;
```

OUTPUT

implicit_add	implicit_mul
15	7

7. DATE: current date and time functions

INPUT (SQL)

```
SELECT CURDATE() AS today, CURTIME() AS time_now, NOW() AS date_time_now;
```

OUTPUT

today	time_now	date_time_now
2026-10-09	16:18:20	2026-10-09 16:18:20

8. DATE: extract parts of a date

INPUT (SQL)

```
SELECT order_id, order_date,  
       YEAR(order_date) AS yr,  
       MONTH(order_date) AS mon,  
       DAY(order_date) AS dy,  
       MONTHNAME(order_date) AS month_name,  
       DAYNAME(order_date) AS day_name,  
       QUARTER(order_date) AS qtr  
FROM orders;
```

OUTPUT

order_id	order_date	yr	mon	dy	month_name	day_name	qtr
1	2026-01-15	2026	1	15	January	Thursday	1
2	2026-03-02	2026	3	2	March	Monday	1
3	2026-07-28	2026	7	28	July	Tuesday	3
4	2026-10-05	2026	10	5	October	Monday	4

9. DATE: EXTRACT, WEEK and DAYOFYEAR

INPUT (SQL)

```
SELECT order_date,
       EXTRACT(YEAR FROM order_date) AS yr,
       WEEK(order_date) AS week_no,
       DAYOFYEAR(order_date) AS day_of_year
FROM orders;
```

OUTPUT

order_date	yr	week_no	day_of_year
2026-01-15	2026	2	15
2026-03-02	2026	9	61
2026-07-28	2026	30	209
2026-10-05	2026	40	278

10. DATE: DATE_ADD, DATE_SUB and LAST_DAY

INPUT (SQL)

```
SELECT order_date,
       DATE_ADD(order_date, INTERVAL 10 DAY) AS plus_10_days,
       DATE_ADD(order_date, INTERVAL 2 MONTH) AS plus_2_months,
       DATE_SUB(order_date, INTERVAL 1 YEAR) AS minus_1_year,
       LAST_DAY(order_date) AS month_end
FROM orders;
```

OUTPUT

order_date	plus_10_days	plus_2_months	minus_1_year	month_end
2026-01-15	2026-01-25	2026-03-15	2025-01-15	2026-01-31
2026-03-02	2026-03-12	2026-05-02	2025-03-02	2026-03-31
2026-07-28	2026-08-07	2026-09-28	2025-07-28	2026-07-31
2026-10-05	2026-10-15	2026-12-05	2025-10-05	2026-10-31

11. DATE: DATEDIFF and TIMESTAMPDIFF - difference between dates

INPUT (SQL)

```
SELECT order_id, order_date,
       DATEDIFF('2026-12-31', order_date) AS days_to_year_end,
       TIMESTAMPDIFF(MONTH, order_date, '2026-12-31') AS months_to_year_end
FROM orders;
```

OUTPUT

order_id	order_date	days_to_year_end	months_to_year_end
1	2026-01-15	350	11
2	2026-03-02	304	9
3	2026-07-28	156	5
4	2026-10-05	87	2

12. DATE: age calculation and filtering by date

INPUT (SQL)

```
SELECT TIMESTAMPDIFF(YEAR, '2005-06-20', '2026-10-09') AS age_in_years;  
SELECT * FROM orders WHERE YEAR(order_date) = 2026 AND MONTH(order_date) >= 7;
```

OUTPUT

```
+-----+  
| age_in_years |  
+-----+  
|          21 |  
+-----+  
  
+-----+-----+-----+-----+  
| order_id | customer | amount | order_date |  
+-----+-----+-----+-----+  
|         3 | Meera    | 2300.00 | 2026-07-28 |  
|         4 | Karan    | 450.25  | 2026-10-05 |  
+-----+-----+-----+-----+
```

LAB 5: Implement SQL Operators (Arithmetic, Comparison and Logical)

Aim: To demonstrate arithmetic, comparison and logical operators in SQL with examples.

1. Create database, table and insert data

INPUT (SQL)

```
DROP DATABASE IF EXISTS lab5;
CREATE DATABASE lab5;
USE lab5;
CREATE TABLE products (
    pid      INT PRIMARY KEY,
    pname    VARCHAR(20),
    category VARCHAR(20),
    price    INT,
    qty      INT
);
INSERT INTO products VALUES
(1, 'Keyboard', 'Accessories', 800, 25),
(2, 'Mouse', 'Accessories', 450, 40),
(3, 'Monitor', 'Display', 9500, 10),
(4, 'Laptop', 'Computer', 55000, 5),
(5, 'Webcam', 'Accessories', 2200, 0),
(6, 'Printer', 'Office', 7200, 8);
SELECT * FROM products;
```

OUTPUT

pid	pname	category	price	qty
1	Keyboard	Accessories	800	25
2	Mouse	Accessories	450	40
3	Monitor	Display	9500	10
4	Laptop	Computer	55000	5
5	Webcam	Accessories	2200	0
6	Printer	Office	7200	8

2. ARITHMETIC operators: + - * / % and DIV

INPUT (SQL)

```
SELECT 20 + 6 AS addition, 20 - 6 AS subtraction, 20 * 6 AS multiplication,
       20 / 6 AS division, 20 DIV 6 AS integer_division, 20 % 6 AS modulus;
```

OUTPUT

addition	subtraction	multiplication	division	integer_division	modulus
26	14	120	3.3333	3	2

3. ARITHMETIC operators applied on table columns

INPUT (SQL)

```
SELECT pname, price, qty,
       price * qty      AS stock_value,
       price + 100      AS price_plus_100,
       price - price*0.10 AS after_10pct_discount,
       price % 1000     AS remainder
FROM products;
```

OUTPUT

pname	price	qty	stock_value	price_plus_100	after_10pct_discount	remainder
Keyboard	800	25	20000	900	720.00	800
Mouse	450	40	18000	550	405.00	450
Monitor	9500	10	95000	9600	8550.00	500
Laptop	55000	5	275000	55100	49500.00	0
Webcam	2200	0	0	2300	1980.00	200
Printer	7200	8	57600	7300	6480.00	200

4. COMPARISON operators: = > <

INPUT (SQL)

```
SELECT pname, price FROM products WHERE category = 'Accessories';
SELECT pname, price FROM products WHERE price > 5000;
SELECT pname, qty FROM products WHERE qty < 10;
```

OUTPUT

pname	price
Keyboard	800
Mouse	450
Webcam	2200

pname	price
Monitor	9500
Laptop	55000
Printer	7200

pname	qty
Laptop	5
Webcam	0
Printer	8

5. COMPARISON operators: >= <= != (<>)

INPUT (SQL)

```
SELECT pname, price FROM products WHERE price >= 7200;
SELECT pname, price FROM products WHERE price <= 800;
SELECT pname, category FROM products WHERE category <> 'Accessories';
```

OUTPUT

pname	price
Monitor	9500
Laptop	55000
Printer	7200

pname	price
Keyboard	800
Mouse	450

pname	category
Monitor	Display
Laptop	Computer
Printer	Office

6. COMPARISON result as 1 (true) / 0 (false) and NULL-safe equal <=>

INPUT (SQL)

```
SELECT 10 > 5 AS gt, 10 = 5 AS eq, 10 != 5 AS ne, NULL = NULL AS null_eq, NULL <=> NULL AS null_safe_eq;
```

OUTPUT

```
+-----+-----+-----+-----+-----+
| gt | eq | ne | null_eq | null_safe_eq |
+-----+-----+-----+-----+
| 1 | 0 | 1 | NULL | 1 |
+-----+-----+-----+-----+
```

7. LOGICAL operator: AND

INPUT (SQL)

```
SELECT pname, category, price FROM products
WHERE category = 'Accessories' AND price > 500;
```

OUTPUT

```
+-----+-----+-----+
| pname | category | price |
+-----+-----+-----+
| Keyboard | Accessories | 800 |
| Webcam | Accessories | 2200 |
+-----+-----+-----+
```

8. LOGICAL operator: OR

INPUT (SQL)

```
SELECT pname, category, price FROM products
WHERE category = 'Display' OR category = 'Office';
```

OUTPUT

```
+-----+-----+-----+
| pname | category | price |
+-----+-----+-----+
| Monitor | Display | 9500 |
| Printer | Office | 7200 |
+-----+-----+-----+
```

9. LOGICAL operator: NOT

INPUT (SQL)

```
SELECT pname, category FROM products
WHERE NOT category = 'Accessories';
```

OUTPUT

```
+-----+-----+
| pname | category |
+-----+-----+
| Monitor | Display |
| Laptop | Computer |
| Printer | Office |
+-----+-----+
```

10. LOGICAL operators combined with brackets and XOR

INPUT (SQL)

```
SELECT pname, category, price, qty FROM products
WHERE (category = 'Accessories' OR category = 'Office') AND qty > 0;
SELECT pname, price, qty FROM products
WHERE (price > 5000) XOR (qty > 8);
```

OUTPUT

```

+-----+-----+-----+
| pname | category | price | qty |
+-----+-----+-----+
| Keyboard | Accessories | 800 | 25 |
| Mouse | Accessories | 450 | 40 |
| Printer | Office | 7200 | 8 |
+-----+-----+-----+
+-----+-----+-----+
| pname | price | qty |
+-----+-----+-----+
| Keyboard | 800 | 25 |
| Mouse | 450 | 40 |
| Laptop | 55000 | 5 |
| Printer | 7200 | 8 |
+-----+-----+-----+

```

LAB 6: Implement SQL Operators (Special Operators and Set Operations)

Aim: To demonstrate special operators (BETWEEN, IN, LIKE, IS NULL, EXISTS, ANY/ALL) and the UNION set operation.

1. Create database, tables and insert data

INPUT (SQL)

```
DROP DATABASE IF EXISTS lab6;
CREATE DATABASE lab6;
USE lab6;
CREATE TABLE students (
    roll_no INT PRIMARY KEY,
    name VARCHAR(20),
    city VARCHAR(20),
    marks INT,
    email VARCHAR(30)
);
INSERT INTO students VALUES
(1, 'Sparsh', 'Bangalore', 82, 'sparsh@mail.com'),
(2, 'Aman', 'Udaipur', 74, NULL),
(3, 'Riya', 'Delhi', 91, 'riya@mail.com'),
(4, 'Neha', 'Bangalore', 68, NULL),
(5, 'Karan', 'Mumbai', 55, 'karan@mail.com');
CREATE TABLE sports (name VARCHAR(20), sport VARCHAR(20));
INSERT INTO sports VALUES ('Sparsh', 'Chess'), ('Riya', 'Cricket'), ('Zoya', 'Tennis'), ('Aman', 'Chess');
SELECT * FROM students;
SELECT * FROM sports;
```

OUTPUT

```
+-----+-----+-----+-----+-----+
| roll_no | name   | city   | marks | email           |
+-----+-----+-----+-----+-----+
| 1       | Sparsh | Bangalore | 82    | sparsh@mail.com |
| 2       | Aman   | Udaipur  | 74    | NULL            |
| 3       | Riya   | Delhi    | 91    | riya@mail.com   |
| 4       | Neha   | Bangalore | 68    | NULL            |
| 5       | Karan  | Mumbai   | 55    | karan@mail.com  |
+-----+-----+-----+-----+-----+
+-----+-----+
| name   | sport  |
+-----+-----+
| Sparsh | Chess  |
| Riya   | Cricket |
| Zoya   | Tennis |
| Aman   | Chess  |
+-----+-----+
```

2. SPECIAL operator: BETWEEN

INPUT (SQL)

```
SELECT name, marks FROM students WHERE marks BETWEEN 70 AND 90;
SELECT name, marks FROM students WHERE marks NOT BETWEEN 70 AND 90;
```

OUTPUT

```
+-----+-----+
| name   | marks |
+-----+-----+
| Sparsh | 82    |
| Aman   | 74    |
+-----+-----+
+-----+-----+
| name   | marks |
+-----+-----+
| Riya   | 91    |
| Neha   | 68    |
| Karan  | 55    |
+-----+-----+
```

3. SPECIAL operator: IN / NOT IN

INPUT (SQL)

```
SELECT name, city FROM students WHERE city IN ('Bangalore','Delhi');  
SELECT name, city FROM students WHERE city NOT IN ('Bangalore','Delhi');
```

OUTPUT

```
+-----+-----+  
| name  | city    |  
+-----+-----+  
| Sparsh | Bangalore |  
| Riya   | Delhi    |  
| Neha   | Bangalore |  
+-----+-----+  
+-----+-----+  
| name  | city    |  
+-----+-----+  
| Aman   | Udaipur  |  
| Karan  | Mumbai   |  
+-----+-----+
```

4. SPECIAL operator: LIKE with % and _ wildcards

INPUT (SQL)

```
SELECT name FROM students WHERE name LIKE 'S%';  
SELECT name FROM students WHERE name LIKE '%a';  
SELECT name FROM students WHERE name LIKE '_i%';
```

OUTPUT

```
+-----+  
| name  |  
+-----+  
| Sparsh |  
+-----+  
+-----+  
| name  |  
+-----+  
| Riya  |  
| Neha  |  
+-----+  
+-----+  
| name  |  
+-----+  
| Riya  |  
+-----+
```

5. SPECIAL operator: IS NULL / IS NOT NULL

INPUT (SQL)

```
SELECT name, email FROM students WHERE email IS NULL;  
SELECT name, email FROM students WHERE email IS NOT NULL;
```

OUTPUT

```
+-----+-----+  
| name  | email  |  
+-----+-----+  
| Aman  | NULL   |  
| Neha  | NULL   |  
+-----+-----+  
+-----+-----+  
| name  | email          |  
+-----+-----+  
| Sparsh | sparsh@mail.com |  
| Riya   | riya@mail.com   |  
| Karan  | karan@mail.com   |  
+-----+-----+
```

6. SPECIAL operator: EXISTS

INPUT (SQL)

```
SELECT name FROM students s
WHERE EXISTS (SELECT 1 FROM sports p WHERE p.name = s.name);
```

OUTPUT

```
+-----+
| name  |
+-----+
| Sparsh|
| Aman  |
| Riya  |
+-----+
```

7. SPECIAL operators: ANY and ALL

INPUT (SQL)

```
SELECT name, marks FROM students
WHERE marks > ALL (SELECT marks FROM students WHERE city = 'Bangalore');
SELECT name, marks FROM students
WHERE marks > ANY (SELECT marks FROM students WHERE city = 'Bangalore');
```

OUTPUT

```
+-----+-----+
| name | marks |
+-----+-----+
| Riya |    91 |
+-----+-----+
+-----+-----+
| name | marks |
+-----+-----+
| Sparsh |    82 |
| Aman   |    74 |
| Riya   |    91 |
+-----+-----+
```

8. SET operation: UNION (removes duplicates)

INPUT (SQL)

```
SELECT name FROM students
UNION
SELECT name FROM sports;
```

OUTPUT

```
+-----+
| name  |
+-----+
| Sparsh|
| Aman  |
| Riya  |
| Neha  |
| Karan |
| Zoya  |
+-----+
```

9. SET operation: UNION ALL (keeps duplicates)

INPUT (SQL)

```
SELECT name FROM students
UNION ALL
SELECT name FROM sports;
```

OUTPUT

```

+-----+
| name  |
+-----+
| Sparsh |
| Aman  |
| Riya  |
| Neha  |
| Karan  |
| Sparsh |
| Riya  |
| Zoya  |
| Aman  |
+-----+

```

10. SET operation: UNION with different columns and ORDER BY

INPUT (SQL)

```

SELECT name, 'Student' AS source FROM students WHERE marks >= 80
UNION
SELECT name, sport FROM sports
ORDER BY name;

```

OUTPUT

```

+-----+-----+
| name  | source |
+-----+-----+
| Aman  | Chess  |
| Riya  | Student|
| Riya  | Cricket|
| Sparsh| Student|
| Sparsh| Chess  |
| Zoya  | Tennis |
+-----+-----+

```

11. SET operations: INTERSECT and EXCEPT (extra)

INPUT (SQL)

```

SELECT name FROM students
INTERSECT
SELECT name FROM sports;
SELECT name FROM students
EXCEPT
SELECT name FROM sports;

```

OUTPUT

```

+-----+
| name  |
+-----+
| Sparsh |
| Aman  |
| Riya  |
+-----+
+-----+
| name  |
+-----+
| Neha  |
| Karan  |
+-----+

```